🎬 Tailwind CSS Mastery Course

✅ Prerequisites: Basic HTML & CSS knowledge

🧠 PART 1: Introduction

- What is Tailwind CSS?
- Why Tailwind CSS over Bootstrap or Custom CSS?
- CDN vs. Install via PostCSS/Vite/CRA
- Real-world usage (e.g. Tailwind used by GitHub, Vercel)

🎥 **Demo:** Basic Tailwind setup using CDN

🧱 PART 2: Project Setup

- ✅ Project 1: Simple HTML + Tailwind (CDN based)
- ✅ Project 2: Vite + Tailwind setup
- ✅ Project 3: React.js + Tailwind CSS integration

🎥 **Demo:** Setup Tailwind in Vite + React

🎨 PART 3: Utility-First Styling

- Understanding Tailwind utility classes
- Customizing Typography
- Colors, Spacing (margin, padding), Sizing
- Fonts, Shadows, Borders, Rounded Corners

🎥 **Demo:** Create a pricing card layout with utilities

🧰 PART 4: Responsive Design

- Mobile-first approach
- `sm`, `md`, `lg`, `xl`, `2xl` breakpoints
- Hide/show based on screen size

🎥 **Demo:** Create a responsive navbar using Tailwind

✨ PART 5: Flexbox & Grid with Tailwind

- `flex`, `justify-`, `items-`, `gap-`, `flex-wrap`
- `grid`, `grid-cols`, `gap-x`, `gap-y`, `place-items`

🎥 **Demo:** Flexbox card layout & grid photo gallery

🧩 PART 6: Forms & Inputs (15 mins)

- Styling inputs, textareas, checkboxes
- Focus states, disabled states
- Form layout using `grid` or `flex`

🎥 **Demo:** Build a styled contact form

🧠 Part 1: Introduction to Tailwind CSS

✅ What is Tailwind CSS?

Tailwind CSS is a modern **utility-first CSS framework**.
It means: instead of writing long custom CSS, you can style your HTML elements directly using pre-defined classes like:

html
```
<div class="bg-blue-500 text-white p-4 rounded-lg">
  Hello Tailwind!
</div>
```

🟢 With Tailwind:

- You don't have to switch between CSS and HTML files
- You can build **beautiful, responsive designs** faster
- Your code becomes more readable and consistent

✅ Why Tailwind CSS (vs Bootstrap or Custom CSS)?

🔍 Feature	Tailwind CSS	Bootstrap	Custom CSS
Design Flexibility	🔥 High	🚫 Limited	✅ Full
Speed of Development	🚀 Fast	🟢 Fast	🟡 Moderate
Customization	💯 Fully Custom	😕 Needs override	💯 Fully Custom
File Size (Build)	📉 Small (purged)	🟥 Large	Depends

🗣️ **In simple terms**:
Tailwind gives you building blocks to design **your own style**, while Bootstrap gives pre-made styles that often look the same on every site.

✅ How to Use Tailwind CSS (CDN vs Install)

🔹 Option 1: Tailwind CSS via CDN (Quick Start)

Use this for:

- Practicing Tailwind
- Small demos and projects
- No need to install anything

📄 Create a file called `index.html` and paste this code:

html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Tailwind CDN Setup</title>
  <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100 p-6">
  <h1 class="text-3xl font-bold text-blue-600">Tailwind is
Working!</h1>
  <p class="text-gray-700 mt-2">This is a basic setup using the
Tailwind CDN.</p>
</body>
</html>
```

📌 Now just **open this file in your browser** — you'll see styled content using Tailwind!

🔹 **Option 2: Tailwind CSS via Install (Vite/React/Advanced Projects)**

Use this for:

- React, Vite, or big projects
- Full control with configuration
- Clean production-ready output
-

🛠️ Installation steps using **Vite**:

bash
```
npm create vite@latest my-tailwind-app
cd my-tailwind-app
npm install

npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

🔧 Update `tailwind.config.js`:

js
```
content: ["./index.html", "./src/**/*.{js,ts,jsx,tsx}"]
```

🧵 Then in your `index.css`:

css
```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

Now you're ready to write Tailwind code inside your Vite or React app.

✅ Real-World Usage of Tailwind CSS

Tailwind CSS is used by many **top tech companies**:

- 💼 **GitHub** → for documentation pages
- 🚀 **Vercel** → homepage and UI
- 🧠 **Laravel** ecosystem
- ✍️ **Hashnode**, **Statamic**, **Raycast**, and many more

📢 So yes, Tailwind is **industry-standard and job-ready**!

🎥 DEMO: Basic Tailwind Setup using CDN (Step-by-Step)

1. Create a file: `index.html`
2. Paste the following code:

html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
  <title>Tailwind Demo</title>
  <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100 min-h-screen flex items-center
justify-center">
  <div class="bg-white p-8 rounded-lg shadow-md">
    <h1 class="text-3xl font-bold text-blue-600 mb-4">Tailwind CSS is
Working!</h1>
    <p class="text-gray-700">Welcome to your first Tailwind demo using
CDN.</p>
  </div>
```

```
</body>
</html>
```

3. Open it in your browser
4. You'll see a beautiful card with styling done **only using Tailwind CSS**

✅ Summary (End of Part 1)

📌 You've learned:

- What Tailwind CSS is
- Why developers prefer Tailwind over Bootstrap
- How to use Tailwind via **CDN (simple)** or **install (advanced)**
- How top companies use Tailwind in production

🧱 PART 2: Project Setup

In this part, we'll learn **how to set up Tailwind CSS** in 3 different ways:

- ✅ **Project 1:** HTML + Tailwind (CDN)
- ✅ **Project 2:** Vite + Tailwind CSS
- ✅ **Project 3:** React.js + Tailwind CSS
- 🎥 Plus a practical demo for each setup

✅ PROJECT 1: Simple HTML + Tailwind (CDN Based)

This is the **quickest way** to start using Tailwind — perfect for small projects or beginners.

🔧 **Steps:**

1. Create a file called `index.html`
2. Paste the below code:

html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Tailwind CDN Setup</title>
  <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100 flex justify-center items-center
min-h-screen">
  <div class="bg-white p-8 rounded-lg shadow-lg">
    <h1 class="text-2xl font-bold text-blue-600">Tailwind CSS
(CDN)</h1>
    <p class="mt-2 text-gray-600">Setup completed using CDN!</p>
  </div>
</body>
</html>
```

3. Save the file and open it in your browser

✅ If you see a styled box — Tailwind is working!

✅ PROJECT 2: Vite + Tailwind CSS Setup

Vite is a super fast build tool (recommended for modern JS projects). This is perfect for professional frontend apps.

🔧 Steps:

1. **Create Vite App**

bash
```
npm create vite@latest tailwind-vite-app
cd tailwind-vite-app
npm install
```

2. **Install Tailwind CSS**

bash
```
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

3. **Update Tailwind Config**
   Inside `tailwind.config.js`:

js
```
content: ["./index.html", "./src/**/*.{js,ts,jsx,tsx}"]
```

4. **Setup Tailwind in CSS**
   Create `src/index.css` and add:

css
```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

5. **Import in `main.jsx` or `main.js`**

js
```
import './index.css';
```

6. **Run your project**

bash
```
npm run dev
```

✅ Tailwind is now fully working in your Vite project!

✅ PROJECT 3: React.js + Tailwind CSS Integration

Let's now set up Tailwind inside a React app using **Create React App (CRA)**:

🔧 **Steps:**

1. **Create React App**

bash
```
npx create-react-app tailwind-react-app
cd tailwind-react-app
```

2. **Install Tailwind**

bash
```
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

3. **Update Config**
   In `tailwind.config.js`:

js
```
content: ["./src/**/*.{js,jsx,ts,tsx}"]
```

4. **Add Tailwind to `src/index.css`**

css
```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

5. **Make sure to import in `index.js`**

js
```
import './index.css';
```

6. **Start the React App**

bash
```
npm start
```

✅ You can now use Tailwind classes in your React components like this:

jsx
```
function App() {
  return (
    <div className="flex justify-center items-center min-h-screen
bg-gray-100">
      <h1 className="text-3xl font-bold text-blue-600">Hello from
Tailwind + React!</h1>
    </div>
  );
}
```

🎥 DEMO: Setup Tailwind in Vite + React

📹 You can now record this as a **screen demo**, where you:

1. Create a Vite React app (`npm create vite@latest`)
2. Choose `React` + `JavaScript`
3. Follow the above Tailwind steps
4. Build a simple layout (e.g., a hero section or card)

💡 **Tip**: Keep a split screen — code on left, output on right.

✅ Summary (End of Part 2)

- We learned 3 ways to set up Tailwind:
    - Simple HTML via CDN
    - Vite + Tailwind for modern frontend apps
    - React.js + Tailwind for component-based UI

🎨 PART 3: Utility-First Styling (Detailed Breakdown)

✅ What Are Utility Classes in Tailwind?

Utility classes are **single-purpose CSS classes** that do one thing — apply a specific style to an element.

Instead of writing this in CSS:

css

```
.card {
  padding: 1rem;
  background-color: white;
```

```
  border-radius: 8px;
}
```

You write it **directly in HTML** like this:

html
```
<div class="p-4 bg-white rounded-lg">Content here</div>
```

👉 This makes development faster, removes the need for custom CSS files, and helps you **build designs visually**.

✅ Spacing: Margin (`m`) and Padding (`p`)

Tailwind provides utility classes for **spacing**, which includes:

Property	Prefix
Padding	`p`
Margin	`m`

You can also apply spacing in specific directions:

Direction	Prefix Example
Top	`mt-4`
Bottom	`mb-2`
Left	`ml-6`
Right	`mr-1`
Horizontal (X)	`mx-4`
Vertical (Y)	`my-3`

🔍 Example: `m-4`

- `m` stands for **margin**
- `4` is a **spacing scale**, which equals `1rem` or `16px`
  (`1 = 0.25rem`, `2 = 0.5rem`, `4 = 1rem`, `8 = 2rem`, etc.)

🧪 So:

html
```
<div class="m-4">This box has 16px margin on all sides</div>
```

✅ Typography Utilities

Tailwind offers many classes for **text styling**, including:

🔸 `text-*` → Font size

html
```
<p class="text-sm">Small text</p> <!-- ~14px -->
<p class="text-xl">Extra large text</p> <!-- ~20px -->
<p class="text-3xl">Very large text</p> <!-- ~30px -->
```

🔸 `font-*` → Font weight

html
```
<p class="font-thin">Thin</p>
<p class="font-semibold">Semi-bold</p>
<p class="font-bold">Bold</p>
```

🔸 `text-gray-600` → Text color

- Tailwind uses color + shade scale
- `gray-600` is a dark gray
- `text-red-500`, `text-blue-700`, `text-green-400` etc.

✅ `tracking-wide` → Letter Spacing

This class controls the **space between letters**.

Class	Spacing Description
`tracking-tight`	Less space between letters
`tracking-normal`	Default
`tracking-wide`	More space between letters

🧪 Example:

html

```
<p class="tracking-wide">W I D E</p>
```

🔍 “`tracking-wide`” adds subtle extra spacing for better readability or style.

✅ `leading-relaxed` → Line Height

This class controls the **space between lines** of text (line height).

Class	Line Height Description
`leading-none`	No extra spacing
`leading-tight`	Small spacing
`leading-normal`	Default line height
`leading-relaxed`	Comfortable spacing

`leading-loose`	More relaxed spacing

🧪 Example:

html
```
<p class="leading-relaxed">This is a paragraph with relaxed line spacing.</p>
```

Great for paragraphs and readability!

✅ Border Radius (`rounded-*`)

Tailwind makes it easy to apply **rounded corners**:

Class	Effect
`rounded`	Small rounding
`rounded-md`	Medium rounding
`rounded-lg`	Larger rounding
`rounded-full`	Fully round (like a circle)

🧪 Example:

html
```
<div class="rounded-lg">This box has soft rounded corners</div>
```

✅ Box Shadow (`shadow-*`)

Shadow utilities add depth to your elements:

Class	Description
`shadow-sm`	Small shadow
`shadow`	Default shadow
`shadow-md`	Medium shadow
`shadow-lg`	Large shadow
`shadow-xl`	Extra large shadow

🧪 Example:

html

```
<div class="shadow-lg">This box has a large shadow</div>
```

✅ Borders and Colors

- `border` → Adds a 1px border
- `border-2`, `border-4` → Thicker borders
- `border-gray-300`, `border-blue-500` → Colored borders

🧪 Example:

html

```
<div class="border border-blue-500 p-4">Box with blue border</div>
```

✅ Fonts and Alignment

- `text-left`, `text-center`, `text-right` → Align text
- `font-mono`, `font-sans`, `font-serif` → Font family
- `uppercase`, `lowercase`, `capitalize` → Text transform

🧪 Example:

html
```
<h2 class="text-center uppercase font-bold">This is centered & uppercase</h2>
```

🎥 DEMO: Create a Pricing Card Layout with Utilities

Here's a simple layout you can use to show all these utilities **in one go**:

html
```
<div class="min-h-screen flex items-center justify-center bg-gray-100">
  <div class="bg-white p-8 rounded-xl shadow-lg max-w-sm text-center">
    <h2 class="text-2xl font-bold text-gray-800 tracking-wide mb-2">Pro Plan</h2>
    <p class="text-gray-600 leading-relaxed mb-4">Ideal for developers and startups who want power and performance.</p>

    <div class="text-4xl font-bold text-blue-600 mb-6">₹999<span class="text-sm text-gray-500">/mo</span></div>

    <ul class="text-left text-gray-700 space-y-2 mb-6">
      <li>✅ Unlimited Projects</li>
      <li>✅ 100 GB Storage</li>
      <li>✅ Priority Support</li>
    </ul>
```

```
    <button class="bg-blue-600 hover:bg-blue-700 text-white px-6 py-2
rounded-lg transition duration-300">
      Get Started
    </button>
  </div>
</div>
```

🧠 In this example:

- We used padding (`p-8`), margin (`mb-4`), text styles, shadow, rounded corners, spacing (`space-y-2`)
- You can **visually teach** each utility while building this

---

✅ Summary of This Part

📚 What we learned:

- How Tailwind utility classes work
- Detailed breakdown of spacing, text, font, colors, shadow, and more
- Difference between `m-4`, `tracking-wide`, `leading-relaxed`, `rounded-xl`, etc.
- How to combine them to build beautiful UI — like pricing cards

---

🧰 PART 4: Responsive Design in Tailwind CSS

Tailwind makes building **mobile-first, responsive UIs** incredibly simple using its **breakpoint prefixes**.

Let's break it down step by step 👇

✅ Mobile-First Approach

Tailwind follows a **mobile-first design philosophy**.

This means:

- You write **default styles for mobile first**
- Then override them for larger screens using breakpoints like `sm:`, `md:`, etc.

🧠 Example:

html
```
<p class="text-sm md:text-xl">This text is small on mobile, large on desktop</p>
```

✅ Breakpoints in Tailwind

Tailwind provides built-in **responsive breakpoints**:

Prefix	Min Width	Devices
`sm:`	640px	Mobile (large)
`md:`	768px	Tablets
`lg:`	1024px	Laptops
`xl:`	1280px	Large desktops
`2xl:`	1536px	Very large screens

🔍 Usage:

html
```
<div class="p-4 md:p-8 lg:p-12">
  <!-- Padding increases as screen gets bigger -->
</div>
```

✅ Hide/Show Elements Based on Screen Size

Tailwind provides simple utility classes to **show or hide content** on different devices:

Class	Meaning
`hidden`	Hide on all screen sizes
`sm:hidden`	Hide on small+ screens (640px+)
`md:block`	Show on medium+ screens (768px+)
`lg:hidden`	Hide on large+ screens (1024px+)

🧠 Example:

html

```
<p class="block md:hidden">Visible only on mobile</p>
<p class="hidden md:block">Visible only on desktop</p>
```

This is perfect for showing **mobile menu buttons**, different images for devices, or different layout blocks.

🎥 DEMO: Create a Responsive Navbar Using Tailwind

We'll now build a **responsive navbar** that:

- Shows a full menu on desktop
- Collapses into a hamburger menu on mobile

✅ HTML Code:

html

```
<!-- Responsive Navbar -->
<nav class="bg-white shadow-md px-6 py-4">
  <div class="max-w-7xl mx-auto flex items-center justify-between">
```

```
    <div class="text-2xl font-bold text-blue-600">Brand</div>

    <!-- Desktop Menu -->
    <ul class="hidden md:flex gap-6 text-gray-700 font-medium">
      <li><a href="#" class="hover:text-blue-600">Home</a></li>
      <li><a href="#" class="hover:text-blue-600">About</a></li>
      <li><a href="#" class="hover:text-blue-600">Services</a></li>
      <li><a href="#" class="hover:text-blue-600">Contact</a></li>
    </ul>

    <!-- Mobile Menu Icon -->
    <div class="md:hidden">
      <button id="menu-btn" class="text-3xl">&#9776;</button>
    </div>
  </div>

  <!-- Mobile Menu (Hidden by default) -->
  <div id="mobile-menu" class="md:hidden hidden px-4 mt-2">
    <ul class="flex flex-col gap-4 text-gray-700 font-medium">
      <li><a href="#" class="hover:text-blue-600">Home</a></li>
      <li><a href="#" class="hover:text-blue-600">About</a></li>
      <li><a href="#" class="hover:text-blue-600">Services</a></li>
      <li><a href="#" class="hover:text-blue-600">Contact</a></li>
    </ul>
  </div>
</nav>
```

✅ JS (Optional - For Toggling Mobile Menu)

html

```
<script>
  const menuBtn = document.getElementById('menu-btn');
  const mobileMenu = document.getElementById('mobile-menu');

  menuBtn.addEventListener('click', () => {
    mobileMenu.classList.toggle('hidden');
  });
</script>
```

✅ What We Used:

- `hidden md:flex`: Hide menu on small devices, show on desktop
- `md:hidden`: Show hamburger icon only on mobile
- Responsive spacing: `px-6`, `mt-2`, `gap-6`
- Utility-based layout with Flexbox: `flex`, `justify-between`, `items-center`

✅ Summary

In this part, we learned:

- How Tailwind's **mobile-first** design works
- Use of **responsive prefixes** like `sm:`, `md:`, etc.
- How to **hide/show content** on different devices
- Built a full **responsive navbar** using Tailwind only

✨ PART 5: Flexbox & Grid with Tailwind CSS

Tailwind CSS makes using Flexbox and Grid incredibly easy with its utility-first approach. Let's break this into two parts:

🔹 A. Flexbox in Tailwind

✅ Basic Flex Setup

To make an element a flex container:

html
```
<div class="flex">...</div>
```

All children become **flex items**.

✅ Common Flexbox Utilities

Class	Description
`flex`	Enables flexbox
`flex-row` / `flex-col`	Horizontal (default) or vertical stacking
`justify-*`	Main axis alignment (left to right)
`items-*`	Cross axis alignment (top to bottom)
`gap-*`	Space between items
`flex-wrap`	Allows wrapping to next line

✅ Justify Content (`justify-*`)

Class	Effect
`justify-start`	Left aligned
`justify-center`	Centered
`justify-end`	Right aligned
`justify-between`	Space between items
`justify-around`	Equal space around

`justify-evenly`	Equal space everywhere

🧠 Example:

html
```
<div class="flex justify-between">
  <div>Item 1</div>
  <div>Item 2</div>
</div>
```

✅ Align Items (`items-*`)

Class	Description
`items-start`	Top aligned
`items-center`	Center vertically
`items-end`	Bottom aligned

✅ Demo: Flexbox Card Layout

html
```
<div class="flex flex-wrap gap-6 p-6 bg-gray-100">
  <div class="bg-white shadow-md p-4 rounded-lg w-64">
    <h2 class="text-xl font-bold mb-2">Card 1</h2>
    <p class="text-gray-600">This is some card content.</p>
  </div>
  <div class="bg-white shadow-md p-4 rounded-lg w-64">
    <h2 class="text-xl font-bold mb-2">Card 2</h2>
    <p class="text-gray-600">This is some more content.</p>
  </div>
  <div class="bg-white shadow-md p-4 rounded-lg w-64">
    <h2 class="text-xl font-bold mb-2">Card 3</h2>
    <p class="text-gray-600">Another one!</p>
```

```
  </div>
</div>
```

✅ This layout:

- Uses `flex-wrap` to wrap cards on smaller screens
- Adds space using `gap-6`
- Cards are fixed width with `w-64` (~16rem)

🔸 B. Grid in Tailwind

✅ Basic Grid Setup

To create a grid container:

html
```
<div class="grid grid-cols-3 gap-4">...</div>
```

- `grid-cols-3` = 3 columns
- `gap-4` = spacing between rows and columns

✅ Common Grid Classes

Class	Description
`grid-cols-*`	Sets number of columns
`gap-*`	Sets uniform gap

`gap-x-*` / `gap-y-*`	Sets horizontal/vertical gap
`place-items-center`	Centers items both axes
`col-span-*`	Span across multiple columns

✅ Demo: Responsive Photo Gallery (Grid)

html

```
<div class="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 gap-4 p-6">
  <img src="https://source.unsplash.com/random/400x300"
class="rounded-lg shadow-md" />
  <img src="https://source.unsplash.com/random/401x300"
class="rounded-lg shadow-md" />
  <img src="https://source.unsplash.com/random/402x300"
class="rounded-lg shadow-md" />
  <img src="https://source.unsplash.com/random/403x300"
class="rounded-lg shadow-md" />
  <img src="https://source.unsplash.com/random/404x300"
class="rounded-lg shadow-md" />
  <img src="https://source.unsplash.com/random/405x300"
class="rounded-lg shadow-md" />
</div>
```

✅ This layout:

- Uses responsive grid: 1 column on mobile, 2 on tablets, 3 on desktop
- Adds consistent spacing with `gap-4`
- Beautiful image cards with `rounded-lg` and `shadow-md`

✅ Flex vs Grid: When to Use?

Use Case	Use
Horizontal/Vertical layout	Flex
Wrapping cards or nav items	Flex
Complex layouts with rows/cols	Grid
Responsive image galleries	Grid

✅ Summary

🎯 In this part, your audience learned:

- How to use **Flexbox** with `flex`, `justify-*`, `items-*`, `gap-*`
- How to wrap content using `flex-wrap`
- How to use **Grid** with `grid-cols-*`, `gap-x`, `gap-y`, `place-items-center`
- Created 2 real projects:
  - Flexbox Card Layout
  - Grid-based Photo Gallery

🧱 Tailwind CSS 12-Column Grid System — Detailed Guide

Tailwind provides a powerful **utility-first grid system** that is highly customizable, responsive, and easy to use without writing a single line of custom CSS.

🔰 1. Enable Grid in Tailwind

To use Tailwind's grid system, apply the `grid` class to a parent container:

html
```
<div class="grid">...</div>
```

By default, this creates a block-level grid container with **1 column**. You control the number of columns using `grid-cols-*` utilities.

🔢 2. Creating a 12-Column Grid

Tailwind supports up to **12 columns** by default.

html
```
<div class="grid grid-cols-12 gap-4">
  <div class="col-span-4 bg-blue-300">4 Columns</div>
  <div class="col-span-8 bg-blue-500">8 Columns</div>
</div>
```

- `grid-cols-12`: Defines a grid with **12 equal columns**
- `col-span-4`: The child takes **4 columns**
- `col-span-8`: The child takes **8 columns**
- `gap-4`: Adds spacing between grid items

📌 Total of `col-span-*` values in a row should ideally sum to 12 for a full-width layout.

🧭 3. Responsive Grid System (Using Breakpoints)

Tailwind uses **mobile-first breakpoints**, which means:

- Styles are applied to **all screen sizes by default**
- You use `sm:`, `md:`, `lg:`, `xl:`, `2xl:` to target **larger screens**

✅ Breakpoints in Tailwind

Breakpoint	Prefix	Min Width
sm	sm:	640px
md	md:	768px
lg	lg:	1024px
xl	xl:	1280px
2xl	2xl:	1536px

📌 Responsive Column Span Example

html

```
<div class="grid grid-cols-12 gap-4">
  <div class="col-span-12 sm:col-span-6 md:col-span-4 bg-purple-300
p-4 rounded">
    Responsive Card
  </div>
</div>
```

🔍 Explanation:

- On **mobile (<640px)**: Full width (`col-span-12`)
- On **small screens (≥640px)**: Half width (`col-span-6`)
- On **medium screens (≥768px)**: One-third width (`col-span-4`)

This helps you build fully responsive layouts without media queries.

🧱 4. Row and Column Gaps

You can control spacing between items using:

Class	Description
`gap-*`	Applies both row and column gap
`gap-x-*`	Horizontal spacing only
`gap-y-*`	Vertical spacing only

html

```
<div class="grid grid-cols-3 gap-6">
  <div class="bg-blue-200">Item 1</div>
  <div class="bg-blue-300">Item 2</div>
  <div class="bg-blue-400">Item 3</div>
</div>
```

🧭 5. Column Start and End (for offsets)

You can control where a column starts or ends using:

Class	Description
`col-start-*`	Starts the column at given position
`col-end-*`	Ends the column at given position

✅ Offset / Centering Example

html

```
<div class="grid grid-cols-12">
  <div class="col-start-4 col-span-6 bg-green-300 p-4 rounded">
    Centered content (starts from 4th column)
  </div>
</div>
```

🔁 6. Auto-Sizing Columns

If you want columns to size automatically based on their content:

html
```
<div class="grid grid-cols-auto">
  <div class="bg-gray-200 p-2">Auto width</div>
  <div class="bg-gray-400 p-2">Another auto width</div>
</div>
```

📝 You can also use:

- `grid-cols-[auto]`
- `grid-cols-[repeat(auto-fit,minmax(150px,1fr))]` for advanced control (JIT mode)

🖼️ 7. Full Responsive Grid Example

html
```
<div class="grid grid-cols-12 gap-4 p-4">
  <div class="col-span-12 sm:col-span-6 md:col-span-4 bg-pink-300 p-4
rounded">Card 1</div>
  <div class="col-span-12 sm:col-span-6 md:col-span-4 bg-pink-400 p-4
rounded">Card 2</div>
  <div class="col-span-12 sm:col-span-6 md:col-span-4 bg-pink-500 p-4
rounded">Card 3</div>
</div>
```

🎯 This layout adjusts automatically:

- 📱 Mobile: Full width cards

- 📊 Tablet: 2 cards per row
- 🖥️ Desktop: 3 cards per row

✅ Summary Table

Feature	Tailwind Class Example
Define columns	`grid-cols-12`
Span columns	`col-span-6`
Responsive span	`sm:col-span-6` `md:col-span-4`
Offset columns	`col-start-3`, `col-end-8`
Gaps	`gap-4`, `gap-x-6`, `gap-y-2`
Mobile-first design	Default applies to mobile first
Breakpoint utilities	`sm:`, `md:`, `lg:`, `xl:`, `2xl:`

🎁 Bonus Tip: Use `container` Class for Layout Width

html

```
<div class="container mx-auto px-4">
  <!-- Your grid content -->
</div>
```

- `container`: Sets max-width per screen size
- `mx-auto`: Centers the layout
- `px-4`: Adds horizontal padding

🧩 PART 6: Forms & Inputs in Tailwind CSS (15 mins)

Master how to build beautiful, fully responsive, and accessible forms using **Tailwind CSS utilities** — without writing a single line of custom CSS.

🎯 What Your Audience Will Learn

- How to **style form elements** (input, textarea, select, checkbox)
- Handle **focus**, **hover**, and **disabled** states
- Build forms using **Flexbox** or **Grid**
- Full **Contact Form mini-project demo**

🔤 1. Basic Input Styling

Tailwind gives full control using utility classes like `border`, `rounded`, `focus:outline-none`, etc.

html

```
<input type="text" placeholder="Enter your name"
  class="w-full border border-gray-300 rounded-md px-4 py-2
focus:outline-none focus:ring-2 focus:ring-blue-500" />
```

🔍 Key Classes Used:

Class	Meaning

`border`	Adds border
`rounded-md`	Rounded corners
`px-4 py-2`	Padding
`focus:outline-none`	Removes browser default outline
`focus:ring-*`	Adds focus ring

📝 2. Textarea Example

html

```
<textarea rows="4" placeholder="Your message"
  class="w-full border border-gray-300 rounded-md px-4 py-2
focus:outline-none focus:ring-2 focus:ring-purple-500"></textarea>
```

✅ 3. Checkbox Styling

html

```
<label class="inline-flex items-center space-x-2">
  <input type="checkbox" class="form-checkbox text-blue-500 w-5 h-5"
/>
  <span class="text-gray-700">Subscribe to newsletter</span>
</label>
```

You can use native checkboxes or integrate plugins like `@tailwindcss/forms` for enhanced styles.

🚫 4. Disabled State

html

```
<input type="text" disabled value="Disabled input"
  class="w-full border bg-gray-100 text-gray-500 px-4 py-2 rounded-md
cursor-not-allowed" />
```

- `bg-gray-100` → Greyed background
- `cursor-not-allowed` → Shows disabled cursor

🧱 5. Form Layout using Flexbox or Grid

Flex Layout:

html

```
<div class="flex flex-col gap-4">
  <input type="text" placeholder="Name" class="input-style" />
  <input type="email" placeholder="Email" class="input-style" />
</div>
```

Grid Layout:

html

```
<div class="grid grid-cols-1 md:grid-cols-2 gap-4">
  <input type="text" placeholder="First Name" class="input-style" />
  <input type="text" placeholder="Last Name" class="input-style" />
</div>
```

> You can define `.input-style` as a reusable class using Tailwind's `@apply` in a custom CSS file if using PostCSS/Vite.

🎥 Demo: Styled Contact Form (Mini Project)

✅ Final HTML (CDN Based)

html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
```

```
  <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
  <title>Tailwind Contact Form</title>
  <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100 flex items-center justify-center min-h-screen
px-4">
  <form class="bg-white p-8 rounded-lg shadow-md w-full max-w-lg
space-y-6">
    <h2 class="text-2xl font-bold text-center text-gray-800">Contact
Us</h2>

    <div class="grid grid-cols-1 md:grid-cols-2 gap-4">
      <input type="text" placeholder="First Name"
        class="border border-gray-300 rounded-md px-4 py-2
focus:outline-none focus:ring-2 focus:ring-blue-500" />
      <input type="text" placeholder="Last Name"
        class="border border-gray-300 rounded-md px-4 py-2
focus:outline-none focus:ring-2 focus:ring-blue-500" />
    </div>

    <input type="email" placeholder="Your Email"
      class="w-full border border-gray-300 rounded-md px-4 py-2
focus:outline-none focus:ring-2 focus:ring-blue-500" />

    <textarea rows="5" placeholder="Your Message"
      class="w-full border border-gray-300 rounded-md px-4 py-2
focus:outline-none focus:ring-2 focus:ring-purple-500"></textarea>

    <label class="inline-flex items-center space-x-2">
      <input type="checkbox" class="form-checkbox text-blue-500 w-5
h-5" />
      <span class="text-gray-700 text-sm">I agree to the terms and
conditions</span>
    </label>

    <button type="submit"
```

```
      class="w-full bg-blue-600 text-white font-semibold py-2 px-4
rounded-md hover:bg-blue-700 transition">
      Submit
    </button>
  </form>
</body>
</html>
```

🧠 Summary

Feature	Tailwind Utility
Borders	`border`, `border-gray-300`
Rounded	`rounded`, `rounded-md`
Focus	`focus:outline-none`, `focus:ring-2`
Layout	`flex`, `grid`, `gap-*`
Responsive	`md:grid-cols-2`
Button	`bg-*`, `hover:bg-*`, `transition`

💼 Final Project: Responsive SaaS Landing Page (Tailwind CSS Mastery)

✅ **Objective:** Build a complete multi-section SaaS website using Tailwind CSS only (no JS frameworks). It will be:

- Fully **responsive**
- Use **utility-first** Tailwind approach
- Include **grid & flex**, **forms**, **buttons**, **responsive nav**, and more

```
<!DOCTYPE html>
<html lang="en" class="dark">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Dark Mode SaaS Landing Page</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <script>
    tailwind.config = {
      darkMode: 'class',
    };
  </script>
</head>
<body class="bg-gray-900 text-gray-100">

  <!-- Navbar -->
  <nav class="bg-gray-800 px-6 py-4 flex items-center justify-between sticky top-0 z-50 text-white">
    <h1 class="text-2xl font-bold text-blue-400">SaaSly</h1>
    <ul class="hidden md:flex space-x-6">
      <li><a href="#features" class="hover:text-blue-400">Features</a></li>
      <li><a href="#pricing" class="hover:text-blue-400">Pricing</a></li>
      <li><a href="#contact" class="hover:text-blue-400">Contact</a></li>
    </ul>
    <button class="md:hidden text-2xl text-white">☰</button>
  </nav>

  <!-- Hero Section -->
```

```
  <section class="bg-gray-900 py-12 px-6 flex flex-col-reverse md:flex-row items-center
justify-center gap-8">
    <div class="md:w-1/2 space-y-4 text-center md:text-left">
      <h2 class="text-4xl font-bold text-white">Build your SaaS website faster</h2>
      <p class="text-gray-400">Tailwind CSS lets you build modern, responsive designs without
writing custom CSS.</p>
      <button class="bg-blue-600 text-white px-6 py-2 rounded-md hover:bg-blue-700">Get
Started</button>
    </div>
    <div class="md:w-1/2">
      <img src="https://dummyimage.com/600x400/333/aaa.png&text=SaaS+Hero+Image"
alt="Hero Image" class="rounded-lg shadow-md" />
    </div>
  </section>

  <!-- Features -->
  <section id="features" class="py-12 px-6 bg-gray-800">
    <h2 class="text-3xl font-semibold text-center text-white mb-10">Core Features</h2>
    <div class="grid grid-cols-12 gap-6">
      <div class="col-span-12 md:col-span-6 lg:col-span-4 p-6 bg-gray-700 rounded shadow
text-center">
        <h3 class="font-bold text-lg mb-2 text-white">Fast Performance</h3>
        <p class="text-sm text-gray-300">Optimized for speed and performance.</p>
      </div>
      <div class="col-span-12 md:col-span-6 lg:col-span-4 p-6 bg-gray-700 rounded shadow
text-center">
        <h3 class="font-bold text-lg mb-2 text-white">Responsive Design</h3>
        <p class="text-sm text-gray-300">Mobile-first layout for all devices.</p>
      </div>
      <div class="col-span-12 md:col-span-6 lg:col-span-4 p-6 bg-gray-700 rounded shadow
text-center">
        <h3 class="font-bold text-lg mb-2 text-white">Easy Customization</h3>
        <p class="text-sm text-gray-300">Style with utilities only.</p>
      </div>
    </div>
  </section>

  <!-- Pricing -->
  <section id="pricing" class="py-12 px-6 bg-gray-900">
    <h2 class="text-3xl font-semibold text-center text-white mb-10">Pricing Plans</h2>
    <div class="grid md:grid-cols-3 gap-6">
      <div class="bg-gray-800 p-6 rounded-lg shadow-md text-center">
        <h3 class="text-xl font-bold mb-2 text-white">Basic</h3>
        <p class="text-2xl font-semibold mb-4 text-blue-400">$19/mo</p>
```

```
      <ul class="text-sm text-gray-300 space-y-2 mb-4">
        <li>✔ 1 Project</li>
        <li>✔ Basic Support</li>
      </ul>
      <button class="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700">Choose
Plan</button>
    </div>
    <div class="bg-gray-800 p-6 rounded-lg shadow-md text-center">
      <h3 class="text-xl font-bold mb-2 text-white">Pro</h3>
      <p class="text-2xl font-semibold mb-4 text-blue-400">$49/mo</p>
      <ul class="text-sm text-gray-300 space-y-2 mb-4">
        <li>✔ 10 Projects</li>
        <li>✔ Priority Support</li>
      </ul>
      <button class="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700">Choose
Plan</button>
    </div>
    <div class="bg-gray-800 p-6 rounded-lg shadow-md text-center">
      <h3 class="text-xl font-bold mb-2 text-white">Enterprise</h3>
      <p class="text-2xl font-semibold mb-4 text-blue-400">$99/mo</p>
      <ul class="text-sm text-gray-300 space-y-2 mb-4">
        <li>✔ Unlimited Projects</li>
        <li>✔ 24/7 Support</li>
      </ul>
      <button class="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700">Choose
Plan</button>
    </div>
  </div>
 </section>

 <!-- Contact Form -->
 <section id="contact" class="py-12 px-6 bg-gray-800">
  <h2 class="text-2xl font-bold text-center mb-6 text-white">Contact Us</h2>
  <form class="max-w-2xl mx-auto bg-gray-900 p-6 rounded shadow space-y-4">
    <div class="grid grid-cols-1 md:grid-cols-2 gap-4">
      <input type="text" placeholder="First Name" class="bg-gray-800 border border-gray-700
text-white px-4 py-2 rounded w-full focus:ring-2 focus:ring-blue-500" />
      <input type="text" placeholder="Last Name" class="bg-gray-800 border border-gray-700
text-white px-4 py-2 rounded w-full focus:ring-2 focus:ring-blue-500" />
    </div>
    <input type="email" placeholder="Your Email" class="bg-gray-800 border border-gray-700
text-white px-4 py-2 rounded w-full focus:ring-2 focus:ring-blue-500" />
```

```
      <textarea rows="5" placeholder="Your Message" class="bg-gray-800 border
border-gray-700 text-white px-4 py-2 rounded w-full focus:ring-2
focus:ring-blue-500"></textarea>
      <button class="w-full bg-blue-600 text-white py-2 rounded
hover:bg-blue-700">Submit</button>
    </form>
  </section>

  <!-- Footer -->
  <footer class="bg-gray-900 text-white p-6 mt-12">
    <div class="grid grid-cols-1 md:grid-cols-3 gap-6 text-center md:text-left">
      <div>
        <h3 class="font-bold text-xl mb-2">SaaSly</h3>
        <p class="text-gray-400">© 2025 All rights reserved.</p>
      </div>
      <div>
        <h4 class="font-semibold mb-2">Links</h4>
        <ul class="space-y-1 text-gray-300">
          <li><a href="#">Home</a></li>
          <li><a href="#">Pricing</a></li>
        </ul>
      </div>
      <div>
        <h4 class="font-semibold mb-2">Follow Us</h4>
        <ul class="flex justify-center md:justify-start space-x-4">
          <li><a href="#">🐦</a></li>
          <li><a href="#">📘</a></li>
          <li><a href="#">📸</a></li>
        </ul>
      </div>
    </div>
  </footer>

</body>
</html>
```